

1. Weiss 7.11

//Build heap

142, 543, 123, 65, 453, 879, 572, 434, 111, 242, 811, 102 OK
142, 543, 123, 65, 453, 879, 572, 434, 111, 242, 811, 102 Swap
142, 543, 123, 65, 811, 879, 572, 434, 111, 242, 453, 102
142, 543, 123, 65, 811, 879, 572, 434, 111, 453, 242, 102 Swap
142, 543, 123, 434, 811, 879, 572, 65, 111, 453, 242, 102
142, 543, 123, 434, 811, 879, 572, 65, 111, 453, 242, 102 Swap
142, 543, 879, 434, 811, 123, 572, 65, 111, 453, 242, 102 Swap
142, 543, 879, 434, 811, 123, 572, 65, 111, 453, 242, 102 OK
142, 543, 879, 434, 811, 123, 572, 65, 111, 453, 242, 102 Swap
142, 811, 879, 434, 543, 123, 572, 65, 111, 453, 242, 102 OK
142, 811, 879, 434, 543, 123, 572, 65, 111, 453, 242, 102 Swap
879, 811, 142, 434, 543, 123, 572, 65, 111, 453, 242, 102 Swap
879, 811, 572, 434, 543, 123, 142, 65, 111, 453, 242, 102 Done

//Remove max

102, 811, 572, 434, 543, 123, 142, 65, 111, 453, 242, 879 Swap
811, 102, 572, 434, 543, 123, 142, 65, 111, 453, 242, 879 Swap
811, 543, 572, 434, 102, 123, 142, 65, 111, 453, 242, 879 Swap
811, 543, 572, 434, 453, 123, 142, 65, 111, 102, 242, 879

//Remove max

242, 543, 572, 434, 453, 123, 142, 65, 111, 102, 811, 879 Swap

572, 543, 242, 434, 453, 123, 142, 65, 111, 102, 811, 879 OK

//Remove max

102, 543, 242, 434, 453, 123, 142, 65, 111, 572, 811, 879 Swap

543, 102, 242, 434, 453, 123, 142, 65, 111, 572, 811, 879 Swap

543, 453, 242, 434, 102, 123, 142, 65, 111, 572, 811, 879 OK

//Remove max

111, 453, 242, 434, 102, 123, 142, 65, 543, 572, 811, 879 Swap

453, 111, 242, 434, 102, 123, 142, 65, 543, 572, 811, 879 Swap

453, 434, 242, 111, 102, 123, 142, 65, 543, 572, 811, 879 OK

//Remove max

65, 434, 242, 111, 102, 123, 142, 453, 543, 572, 811, 879 Swap

434, 65, 242, 111, 102, 123, 142, 453, 543, 572, 811, 879 Swap

434, 111, 242, 65, 102, 123, 142, 453, 543, 572, 811, 879 OK

//Remove max

142, 111, 242, 65, 102, 123, 434, 453, 543, 572, 811, 879 Swap

242, 111, 142, 65, 102, 123, 434, 453, 543, 572, 811, 879 OK

//Remove max

123, 111, 142, 65, 102, 242, 434, 453, 543, 572, 811, 879 Swap

142, 111, 123, 65, 102, 242, 434, 453, 543, 572, 811, 879

//Remove max

102, 111, 123, 65, 142, 242, 434, 453, 543, 572, 811, 879 Swap

```

123, 111, 102, 65, 142, 242, 434, 453, 543, 572, 811, 879

//Remove max

65, 111, 102, 123, 142, 242, 434, 453, 543, 572, 811, 879 Swap

111, 65, 102, 123, 142, 242, 434, 453, 543, 572, 811, 879

//Remove max

102, 65, 111, 123, 142, 242, 434, 453, 543, 572, 811, 879 OK

//Remove max

65, 102, 111, 123, 142, 242, 434, 453, 543, 572, 811, 879

//Done

65, 102, 111, 123, 142, 242, 434, 453, 543, 572, 811, 879

```

2. Weiss 7.19

```

3 1 4 1 5 9 2 6 5 3 5 // start

// medianOfThree(3,9,5) -> 5, swap pivot to centre and assign as pivot
3 1 4 1 5 5 2 6 5 3 9
      ^
      p

3 1 4 1 5 3 2 6 5 5 9 // place pivot on the side
              ^
              p

3 1 4 1 5 3 2 6 5 5 9 // set up left and right iterators
^                   ^
i                   jp

3 1 4 1 5 3 2 6 5 5 9 // move left iterator until value >= pivot
      ^           ^
      i           jp

3 1 4 1 5 3 2 6 5 5 9 // move right iterator until value <= pivot
      ^           ^ ^

```

```

        i          j p
3 1 4 1 5 3 2 6 5 5 9 // swap
      ^          ^ ^
        i          j p

3 1 4 1 5 3 2 6 5 5 9 // move left iterator until value >= pivot
              ^ ^ ^
              i j p

3 1 4 1 5 3 2 6 5 5 9 // move right iterator until value <= pivot
              ^ ^ ^
              j i  p

3 1 4 1 5 3 2 6 5 5 9 // no swap because iterators crossed
              ^ ^ ^
              j i  p

3 1 4 1 5 3 2 5 5 6 9 // swap pivot back into location with i
              ^
              p

3 1 4 1 5 3 2          // divide and conquer. First half first
x

// medianOfThree(3,1,2) -> 2, swap and assign as pivot
1 1 4 2 5 3 3
    ^
    p

1 1 4 3 5 2 3          // place pivot on the side
    ^
    p

1 1 4 3 5 2 3          // set up left and right iterators
^          ^
i          jp

1 1 4 3 5 2 3          // move left iterator until value >= pivot
    ^          ^
    i          jp

1 1 4 3 5 2 3          // move right iterator until value <= pivot
    ^ ^          ^
    j i          p

```

```

1 1 4 3 5 2 3      // no swap because iterators crossed
  ^ ^           ^
  j i         p

1 1 2 3 5 4 3      // swap pivot back into location with i
  ^
  p

1 1                // divide and conquer, insertion sort

      3 5 4 3      // divide and conquer

      3 3 4 5      // medianOfThree(3,4,3) -> 3, swap and assign as
pivot      ^
          p

      3 4 3 5      // place pivot on the side
          ^
          p

      3 4 3 5      // set up left and right iterators
      ^ ^
      i   jp

      3 4 3 5      // move left iterator until value >= pivot
      ^ ^
      i jp

      3 4 3 5      // move right iterator until value <= pivot
      ^ ^ ^
      j i p

      3 4 3 5      // no swap because iterators crossed
      ^ ^ ^
      j i p

      3 3 4 5      // swap pivot back into location with i
      ^
      p

      3                // divide and conquer, done

      4 5            // divide and conquer, insertion sort

```

```

5 6 9 // divide and conquer, second half.

5 6 9 // medianOfThree(5,6,9) -> 6
  ^
  p

5 9 6 // place pivot on the side
^ ^ ^
i j p

5 9 6 // move left iterator until value >= pivot
  ^ ^
  ijp

5 9 6 // move right iterator until value <= pivot
^ ^ ^
j i p

5 9 6 // no swap because iterators crossed
^ ^ ^
j i p

5 6 9 // swap pivot back into location with i
  ^
  p

5      // divide and conquer, done

      9 // divide and conquer, done

1 1 2 3 3 4 5 5 5 6 9 // done

```

3. Weiss 7.24

The worst-case scenario for quicksort is when we constantly pick the pivot that divides the array most unevenly. This will reduce the answer to selection sort. Let's aim for picking the highest number in the array at each median-of-three partition.

We want a permutation of 20 numbers. Let's keep an ordered list of the first 20 integers and pick as we go.

```
[18, 3, 6, 7, 10, 11, 14, 1, 16, 19, 5, 12, 4, 13, 8, 15, 2, 17, 9,
20]
```

```
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20
[A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T]
```

```
// Place the highest three numbers to be picked for median.
```

```
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20
[18, B, C, D, E, F, G, H, I, 19, K, L, M, N, O, P, Q, R, S, 20]
```

```
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20
[18, B, C, D, E, F, G, H, I, S, K, L, M, N, O, P, Q, R, 19, 20]
```

^
p

There will be no number in between 18 and 19 that's greater than the pivot. Result is that we do quicksort again.

```
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20
[18, B, C, D, E, F, G, H, 16, S, K, L, M, N, O, P, Q, 17]
```

```
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20
[16, B, C, D, E, F, G, H, Q, S, K, L, M, N, O, P, 17, 18]
```

^
p

```
//pick 15 and 1
```

```
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20
[16, B, C, D, E, F, G, 1, Q, S, K, L, M, N, O, 15]
```

```
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20
[1, B, C, D, E, F, G, 15, Q, S, K, L, M, N, O, 16]
```

```
//assign 15 as pivot
```

```
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20
[1, B, C, D, E, F, G, O, Q, S, K, L, M, N, 15, 16]
```

^
p

```
//there's nothing to swap in between 1 and 15. Move to subarray
```

```
//pick 13 and 14
```

```
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20
[1, B, C, D, E, F, 14, O, Q, S, K, L, M, 13]
```

```
//assign 13 as pivot
```

```
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20
[1, B, C, D, E, F, M, O, Q, S, K, L, 13, 14]
```

^
p

```
//move to subarray
//pick 12 and 11
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20
[1, B, C, D, E, 11, M, O, Q, S, K, 12]
```

```
//assign 11 as pivot
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20
[1, B, C, D, E, K, M, O, Q, S, 11, 12]
      ^
      p
```

```
//move to subarray
//pick 9 and 10
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20
[1, B, C, D, 10, K, M, O, Q, 9]
```

```
//assign 9 as pivot
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20
[1, B, C, D, Q, K, M, O, 9, 10]
      ^
      p
```

```
//subarray
//pick 7 and 8
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20
[1, B, C, 7, Q, K, M, 8]
```

```
//assign 7 as pivot.
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20
[1, B, C, M, Q, K, 7, 8]
      ^
      p
```

```
//subarray
//pick 5 and 6
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20
[1, B, 6, M, Q, 5]
```



```
//assign as pivot
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20
[1, B, Q, M, 5, 6]
      ^
      p
```

```
//subarray
//we are still not at cutoff, which is 3.
//let's pick 3 for B and 4 for M
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20
[1, 3, Q, 4]
```

```
//assign as pivot
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20
[1, Q, 3, 4]
      ^
      p
```

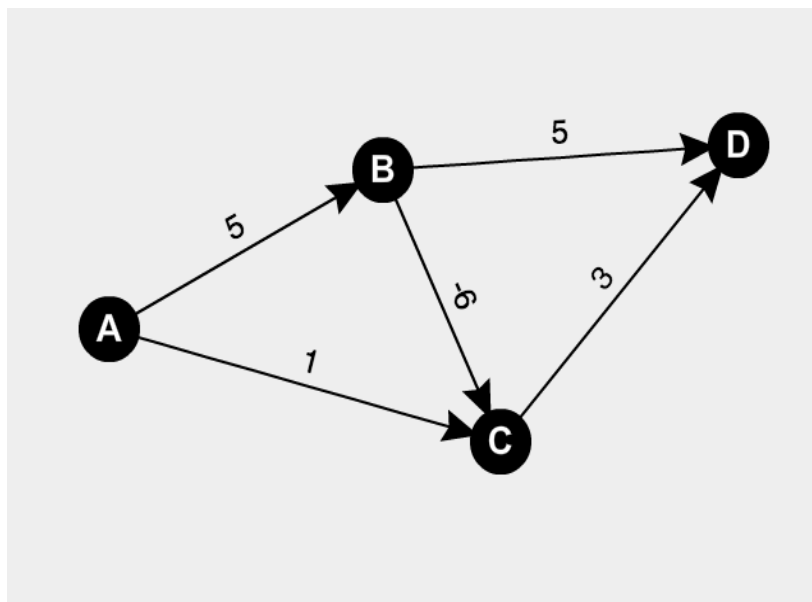
Q will automatically be 2. Now we found our array:
[18, 3, 6, 7, 10, 11, 14, 1, 16, 19, 5, 12, 4, 13, 8, 15, 2, 17, 9, 20]

4.Weiss 9.1

Topological Ordering: S - G - D - A - B - H - E - I - F - C - T

There are several correct orderings outside of this ordering.

5.Weiss 9.7a



Dijkstra starting from A will explore vertices in the following order: C, D, B. In doing so, it will find A->C->D route to D, which has cost 4. It will miss A->B->C->D which has cost 2.

6.Weiss Exercise 9.38

You are given a set of N sticks, which are lying on top of each other in some con-figuration. Each stick is specified by its two endpoints; each endpoint is an ordered triple giving its x, y, and z coordinates; no stick is vertical. A stick may be picked up only if there is no stick on top of it.

a) Explain how to write a routine that takes two sticks a and b and reports whether a is above, below, or unrelated to b. (This has nothing to do with graph theory.)

Project all the sticks to the x-y plane. If the projections of two sticks do not intersect, we know they are unrelated; otherwise, suppose the intersection point of them is (p, q), then we can compute the z-coordinate of each stick by substituting the x, y value of the line equation corresponding to each stick using p, and q respectively, and the stick with a larger z--coordinate value is above the other.

Pseudo-code:

```
//returns the spatial relation between sticks a, b
SpatialRelationSticks(a,b){

    If(projections of a and b to x-y plane do not intersect)
        //get x-y slopes of sticks
        //find point of intersection between sticks=(p,q)
        //sticks intersect if point of intersection is within the bounds
of both sticks
        Return unrelated;

    else{

        compute the z-coordinate of each stick (denoted as z(a) and
z(b)) by
        substituting the x, y value of the line equation
corresponding to each
        stick using p, and q respectively;

        if(z(a)>z(b))
```

```

        Return above;

    Else
        Return below;
}
}

```

b) Give an algorithm that determines whether it is possible to pick up all the sticks, and if so, provides a sequence of stick pickups that accomplishes this.

Construct a directed graph $G = (V, E)$ as follows. Each vertex of V corresponds to one stick. And a directed edge is constructed from vertex a to vertex b , if the corresponding sticks, denoted as $s(a)$ and $s(b)$, satisfy $s(a)$ is above $s(b)$. Then we can run Topological Sort algorithm on G . If there is cycle in G found by Topological Sort algorithm, we know that it is not possible to pick up all the sticks; otherwise, we follow the order of each stick produced by Topological Sort algorithm to pick them up.

Pseudo-code:

```

//returns the order of sticks picked up or if it wasn't possible
PickUpSticks(s(1),s(2),...,s(n)) //s(n)=listofnsticks

    for(each stick s in list s(v))
        Construct a vertex v for Graph G;

    Project all sticks to x-y plane;
    Find all intersections between any two sticks by running the
algorithm from(a)
    between every pair of sticks;

    if(spacial relation between two sticks)
        add directed edge between vertices of two sticks to G;

    G.topsort(); // topsort() method from Addison Wesley textbook

    if cycle is found in G by topsort()
        return No;
    else
        return the order of each stick produced by TopologicalSort;

```